

同济大学

汉诺塔大作业实验报告

2451951 大数据 于沛可

2025/12/15

1. 题目

本次作业要求实现一个完整的汉诺塔演示与游戏系统，包含以下九个功能模块：

1.1.基础递归求解模块（1-2）

实现汉诺塔问题的递归算法-->显示移动步骤和步数统计

1.2.数组可视化模块（3-4）

用数组模拟三个柱子的状态 --> 实现横向和纵向的实时显示 --> 延时演示和单步执行

1.3.图形演示模块（5-8）

在控制台中绘制图形化的汉诺塔 --> 实现盘子的动画移动效果（上移-平移-下移） --> 图形与文字同步显示 --> 支持多种配置参数（正常版/瘦版/胖版）

1.4.交互游戏模块（9）

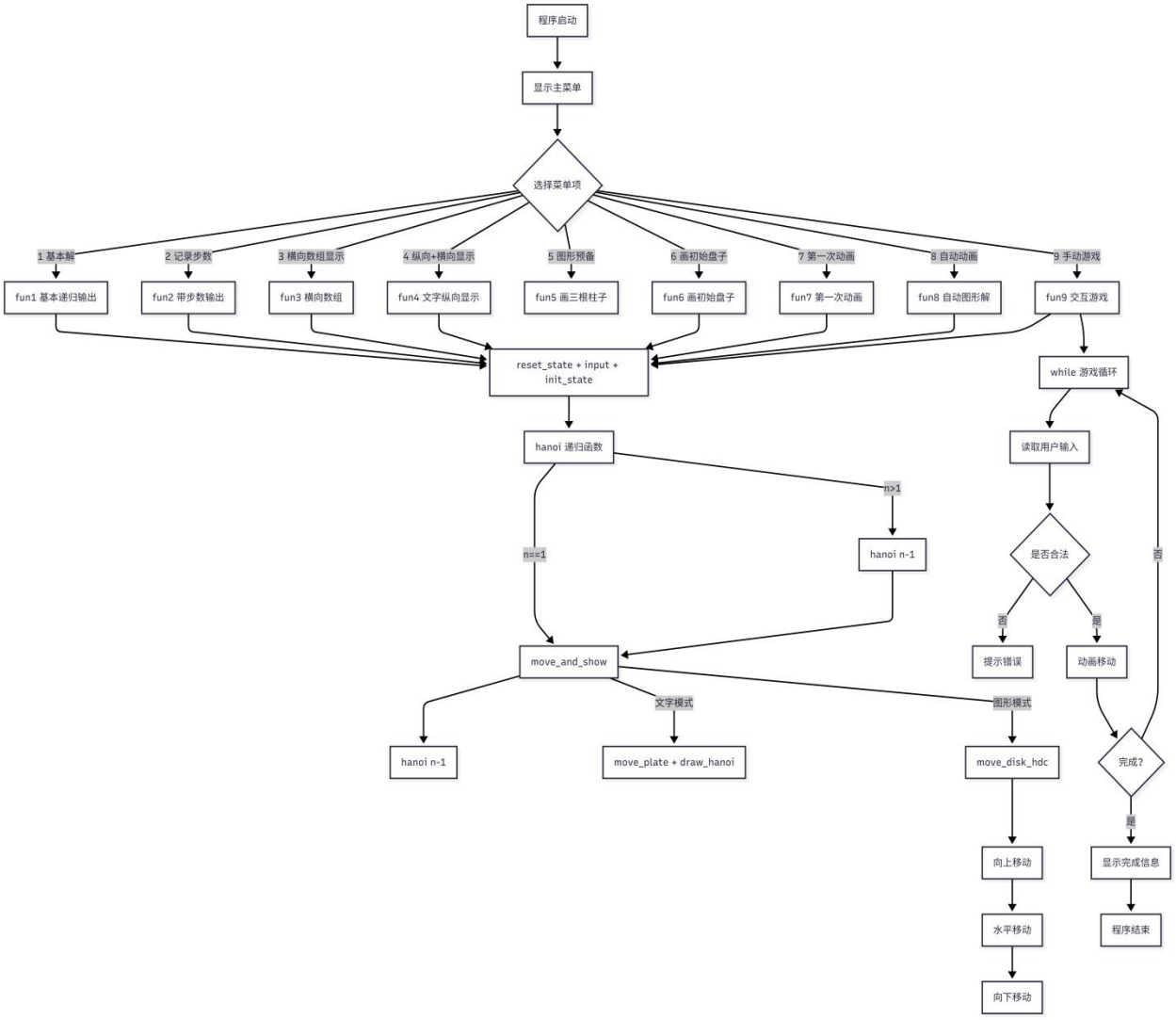
用户手动输入移动指令 --> 检查移动合法性 --> 判断游戏完成并统计步数

1.5.基本要求

1. 不违反白名单（除了处理输入函数可以使用指针）
2. 递归函数不超过15行
3. 通过参数实现函数在不同模式下的调用，调用同一函数
4. 不得使用除必要/允许的全局变量外其他任何全局变量

2. 整体设计思路

整个程序流程图如下：



2.1.程序架构

本次大作业需要自主设计以下四个程序：

1. hanoi_main.cpp：主控制模块

负责初始化屏幕 --> 显示菜单并接收用户选择 --> 调用对应功能函数

2. hanoi_menu.cpp：菜单显示模块

显示9个功能选项 - 返回用户选择

3. hanoi_multiple_solutions.cpp：所有9个功能的具体实现

4. hanoi.h：包含所有额外设计的函数（fun1~9）以及const常量定义的头文件（配置参数）、字符显示工具和图形显示工具的头文件

5. 除此之外，还包括已经提供的cmd_console_tools.h/cmd_console_tools.cpp（字符显示）、cmd_hdc_tools.h/cmd_hdc_console_tools.cpp(图形显示)、hanoi_const_value.h（配置文件，参数可变）

2.2.底层数据设计

本程序使用三个数组和三个栈顶指针模拟汉诺塔状态：

arr_A[10], arr_B[10], arr_C[10]：存储各柱子上的盘子编号

top_A, top_B, top_C：记录各柱子的盘子数量

count_step：全局步数计数器

delay：延时参数

2.3.关键设计思想

1. 参数化设计

一个递归函数通过mode参数支持所有功能：输入、显示等函数通过参数实现复用

2. 状态同步

图形移动和数组更新严格同步，文字显示实时反映当前状态

3. 分层设计

递归层：hanoi()负责算法逻辑；显示层：move_and_show()根据mode调用不同显示；动画层：move_disk_hdc()实现图形动画

3. 主要功能的实现

3.1. 递归函数：

伪代码如下：

```
hanoi(n, src, tmp, dst,mode):
    if n == 1:
        移动盘子从src到dst（并进行图形/文字的显示）
        return
    hanoi(n-1, src, dst, tmp)  // 将n-1个盘子从src借助dst移到tmp
    移动第n个盘子从src到dst（并进行图形/文字的显示）
    hanoi(n-1, tmp, src, dst)  // 将n-1个盘子从tmp借助src移到dst
```

3.2. 实现盘子移动与图形/文字显示函数：

在整个程序中实现底层内部状态的更新函数：move_plate函数，并返回盘子号

```
int move_plate(char src, char dst, int mode)  //返回移动的盘子号
    根据模式（主要是4/8）来计算A/B/C三个柱子的底座坐标
    if mode=4/8:
        进行清除操作：用空格覆盖src最上面一个盘子，用来显示纵向打印动态变化效果
    定义 int plate
    //从src出栈
    if (src == 'A')
        plate = arr_A[--top_A];
    else if (src == 'B')
        plate = arr_B[--top_B];
    else
        plate = arr_C[--top_C];
    // 入栈到dst
    if (dst == 'A')
        arr_A[top_A++] = plate;
    else if (dst == 'B')
        arr_B[top_B++] = plate;
    else
        arr_C[top_C++] = plate;

    count_step++;
    return plate;
```

在hanoi递归函数中调用的，整合底层状态更新和图形/文字显示的函数：move_and_show

```
//根据模式选择，移动并打印，是hanoi递归函数的主体调用部分
move_and_show(char src, char dst, int plate, int mode, int layer)
    if mode=8:
        move_disk_hdc; //显示图形移动
        move_plate; //更新内部数组状态
        同步显示文字部分：
        cct_gotoxy(MenuItem8_Start_X, MenuItem8_Start_Y);
        show_row; //横向显示每一步的移动
        draw_hanoi; //纵向显示每一步的移动
    else if mode = 1/2/3/4:
        各自调用函数，实现各自功能
```

3.3. 图形相关函数：

void draw_three_columns() //画立柱和底座

void draw_init_disk(int layer, char src) //在初始柱子 src 上画初始盘子

```
move_disk_hdc(int plate, char src, char dst, int layer)
    1.计算盘子、src/dst 柱子的位置参数
    2.移动阶段：
        a. 向上移动阶段
        While (cur_y 盘子当前纵坐标 > HDC_Top_Y):
            擦除下方strip
            向上移动 HDC_Step_Y
            画出上方strip
            如在立柱范围内，恢复立柱
            同时执行延时，同步更新盘子坐标

        b. 水平移动阶段
        While (cur_x != dst_x):
            擦除移动方向相反一侧的strip
            水平移动 HDC_Step_X
            画出移动方向一侧的strip
            同时执行延时，同步更新盘子坐标

        c. 向下移动阶段
        （类似向上，方向相反）
```

3.4.文字相关函数：

void show_row(int plate, char src, char dst) //横向显示每一步移动

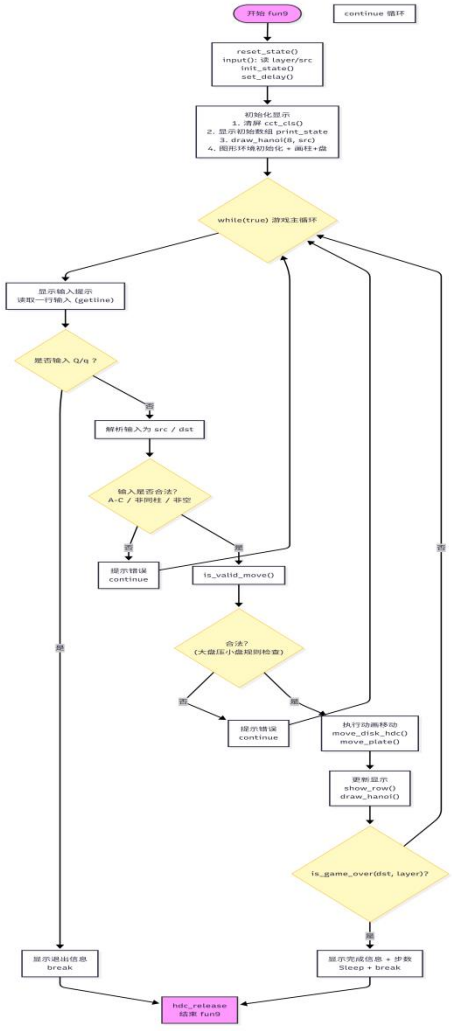
void print_state_row() //横向打印三个柱子的状态

void draw_hanoi(int mode, char src) //打印纵向汉诺塔

void show_col(int plate, char src, char dst) //纵向显示每一步移动

3.5.fun1~fun9：

fun1~fun9各自通过调用以上函数来实现题目要求中1~9的功能，其中第9项功能函数流程图如下：



3.6.其他工具函数：

int input(char* src, char* dst, char* tmp) //输入函数

void set_delay() //设置延时等级

void perform_delay() //执行延时

void reset_state() //初始化清空所有值

void init_state(char src, int layer) //初始化栈

int get_base_x(char col) //计算某个柱子底座左上角的 x 坐标

4. 调试过程碰到的问题

4.1.图形区和文字区重叠导致显示异常

起初，我的盘子在水平移动时会出现"缩短-变长"的错误闪烁效果，无法正确显示。分析原因，我的文字部分如“按回车键继续”定位到了屏幕的顶部，文字区与水平移动的图形区重叠，从而导致无法图形部分无法正确显示。因此，在每次绘图前，我先利用cct_gotoxy()将光标定位到距离图形区下方足够远的地方，从而解决图形显示问题。

4.2.频繁调用cct_gotoxy（）函数

起初，我的draw_hanoi()函数在清除柱子上移动过的盘子这一部分，将整个盘子都用空格覆盖再去全部绘制新状态，从而导致大量调用cct_gotoxy()函数，导致光标在屏幕上乱飞，有时候会飞到图形显示去导致如前所述的图形与文字重叠冲突问题。因此，我在draw_hanoi()中去除了清除的部分并让我的move_plate()函数进行实现，这样可以做到在移动盘子时，只清除源柱顶部的一个盘子，调用cct_gotoxy()函数的频率大量减少，光标也较为固定不再乱飞。

5. 心得体会

通过本次实验，给我带来以下几个方面的收获：

1. 通过本次实验，带给我主要的思考是：状态数据（如 A/B/C 三个柱子的数组和栈顶指针）是整个程序的核心，任何显示或动画都必须严格以这些数据为依据；图形与文字混合输出时，对屏幕坐标的控制非常关键，稍有不慎就会导致显示错乱。同时我也认识到，在一开始设计程序结构时就做好模块划分，能有效降低后期调试和功能扩展的难度。

2. 在完成较复杂程序时，将整体问题分解为若干小功能模块是更合理的做法。

3. 本次实验的一个重要体会是：后续功能是否能够高效完成，很大程度上取决于前期代码是否具有良好的可复用性。让我学会利用参数设计进行函数的复用：设计函数时考虑通用性，增加参数而不是写死：例如，在本题中，递归函数复用：设计hanoi()函数时，通过mode参数支持9种模式；输入函数复用：所有题目共用一个input()函数，通过指针参数返回多个值；显示函数复用：print_state_row()（题3/4/8/9共用）：横向显示数组、draw_hanoi()：纵向显示数组（题4/8共用）、move_disk_hdc()：图形动画（题7/8/9共用）……

4. 复杂程序不是靠写得多，而是靠函数划分是否合理来完成。同时，写代码时要注意给出注释，函数名称最好能看出功能，这样可以省去很多麻烦。

6. 附件：源程序

```
void hanoi(int n, char src, char tmp, char dst,
int mode, int layer)
{
    if (n == 1)
    {
        move_and_show(src, dst, 1, mode,
layer);
        return;
    }
    hanoi(n - 1, src, dst, tmp, mode, layer);
//先将上面的 n-1 个从起始柱移动到中间柱
    move_and_show(src, dst, n, mode, layer);
//将最下面的从起始柱移动到目标柱
    hanoi(n - 1, tmp, src, dst, mode, layer);
//最后将上面的 n-1 个从中间柱移动到目标柱
    }
}

void print_state_row() //横向打印三个柱子的
状态
{
    int i;
    cout << "A:"; //打印 A 柱
    for (i = 0; i < top_A; i++)
        cout << setw(2) << arr_A[i];
    for (i = top_A; i < 10; i++)
        cout << "  ";
    cout << "B:"; //打印 B 柱
    for (i = 0; i < top_B; i++)
        cout << setw(2) << arr_B[i];
    for (i = top_B; i < 10; i++)
        cout << "  ";
    cout << " C:"; //打印 C 柱
    for (i = 0; i < top_C; i++)
        cout << setw(2) << arr_C[i];
    for (i = top_C; i < 10; i++)
        cout << "  ";
}

void show_row(int plate, char src, char dst)
//横向显示每一步移动
```

```
{
    cout << "第" << setw(4) << count_step << "步(" << setw(2) << plate << "): " << src << "-->" << dst << " ";
    print_state_row();
    cout << endl;
}

void move_disk_hdc(int plate, char src, char dst, int layer)
{
    // 获取移动前的层数
    int cur_layer_src;
    if (src == 'A')
        cur_layer_src = top_A;
    else if (src == 'B')
        cur_layer_src = top_B;
    else
        cur_layer_src = top_C;
    int cur_layer_dst;
    if (dst == 'A')
        cur_layer_dst = top_A;
    else if (dst == 'B')
        cur_layer_dst = top_B;
    else
        cur_layer_dst = top_C;
    // 计算盘子参数
    int disk = plate;
    int disk_w = (2 * disk + 1) * HDC_Base_Width;
    int disk_h = HDC_Base_High;
    // 计算柱子中心位置
    int base_width = 23 * HDC_Base_Width;
    int src_base_x = get_base_x(src);
    int dst_base_x = get_base_x(dst);
    int src_center_x = src_base_x + base_width / 2;
    int dst_center_x = dst_base_x + base_width / 2;
    // 盘子当前位置
    int cur_x = src_center_x - disk_w / 2;
    int cur_y = HDC_Start_Y - cur_layer_src * disk_h;
    // 立柱顶部 Y 坐标
    int column_top_y = HDC_Start_Y - 12 * HDC_Base_High;
    //  向上移动
    while (cur_y > HDC_Top_Y)
    {
        // 画上方的新位置
        cur_y -= HDC_Step_Y;
        hdc_rectangle(cur_x, cur_y, disk_w, HDC_Step_Y, HDC_COLOR[disk]);
        // 擦除下方的旧位置
        hdc_rectangle(cur_x, cur_y + disk_h, disk_w, HDC_Step_Y, HDC_COLOR[0]);
        // 如果还在立柱范围内，需要恢复被擦除的立柱中间部分
        if (cur_y > column_top_y - disk_h)
        {
            int col_x = src_center_x - HDC_Base_Width / 2;
            int col_y = cur_y + disk_h;
            hdc_rectangle(col_x, col_y, HDC_Base_Width, HDC_Step_Y, HDC_COLOR[MAX_LAYER + 1]);
        }
        perform_delay();
    }
    // 水平移动
    int dst_x = dst_center_x - disk_w / 2;
    int step_x = (dst_x > cur_x ? HDC_Step_X : -HDC_Step_X);
    while (cur_x != dst_x)
    {
        if (step_x > 0)
        {
            // 向右移动：画右边的新 strip
            hdc_rectangle(cur_x + disk_w, cur_y, HDC_Step_X, disk_h, HDC_COLOR[disk]);
            hdc_rectangle(cur_x, cur_y, HDC_Step_X, disk_h, HDC_COLOR[0]);
            cur_x += step_x;
        }
        else
        {
            // 向左移动：画左边的新 strip
            hdc_rectangle(cur_x + step_x,
```

```
cur_y, -step_x, disk_h, HDC_COLOR[disk]);
            cur_x += step_x;
            hdc_rectangle(cur_x + disk_w,
cur_y, -step_x, disk_h, HDC_COLOR[0]);
        }
        perform_delay();
    }
    // 向下移动
    int dst_y = HDC_Start_Y - (cur_layer_dst
+ 1) * disk_h;
    int dst_column_top_y = HDC_Start_Y - 12 *
HDC_Base_High;
    while (cur_y < dst_y)
    {
        // 画下方的新位置
        hdc_rectangle(cur_x, cur_y + disk_h,
disk_w, HDC_Step_Y, HDC_COLOR[disk]);
        // 擦除上方的旧位置
        hdc_rectangle(cur_x, cur_y, disk_w,
HDC_Step_Y, HDC_COLOR[0]);
        // 如果还在立柱范围内，需要恢复被擦
除的立柱中间部分
        if (cur_y >= dst_column_top_y)
        {
            int col_x = dst_center_x -
HDC_Base_Width / 2;
            int col_y = cur_y;
            hdc_rectangle(col_x, col_y,
HDC_Base_Width, HDC_Step_Y,
HDC_COLOR[MAX_LAYER + 1]);
        }
        cur_y += HDC_Step_Y;
        perform_delay();
    }
}


void move_and_show(char src, char dst, int
plate, int mode, int layer) //根据模式选择，
移动并打印
{
    if (mode == 8)
    {
        move_disk_hdc(plate, src, dst,
layer);
        move_plate(src, dst, 8);
        //同步显示文字部分（横向打印）
        cct_gotoxy(MenuItem8_Start_X,
MenuItem8_Start_Y);
        show_row(plate, src, dst);
        cout << "            "; // 清除行尾
残留
        // 显示纵向数组
        draw_hanoi(8, src);
    }
    else
    {
        if (mode == 1)
        {
            plate = move_plate(src, dst, 1);
            cout << plate << "# " << src <<
"--->" << dst << endl;
        }
        else if (mode == 2)
        {
            plate = move_plate(src, dst, 2);
            cout << setw(4) << count_step <<
":" << plate << "# " << src << "--->" << dst
<< endl;
        }
        else if (mode == 3)
        {
            plate = move_plate(src, dst, 3);
            show_row(plate, src, dst);
        }
        else
        {
            plate = move_plate(src, dst, 4);
            show_col(plate, src, dst);
        }
    }
}


int move_plate(char src, char dst, int mode)
//返回移动的盘子号
{
    int X_A, Y_A, X_B, Y_B, X_C, Y_C;
    if (mode == 4)
    {
```

```
        X_A = MenuItem4_Start_X +
Underpan_A_X_OFFSET;
        Y_A = MenuItem4_Start_Y +
Underpan_A_Y_OFFSET;
        X_B = X_A + Underpan_Distance;
        Y_B = Y_A;
        X_C = X_B + Underpan_Distance;
        Y_C = Y_A;
    }
    else if (mode == 8)
    {
        X_A = MenuItem8_Start_X +
Underpan_A_X_OFFSET;
        Y_A = MenuItem8_Start_Y +
Underpan_A_Y_OFFSET;
        X_B = X_A + Underpan_Distance;
        Y_B = Y_A;
        X_C = X_B + Underpan_Distance;
        Y_C = Y_A;
    }
    if (mode == 4 || mode == 8)
    {
        // 只清除最上面一个盘子
        if (src == 'A' && top_A > 0)
        {
            cct_gotoxy(X_A - 1, Y_A - 2 -
(top_A - 1)); // 定位到最上面盘子的位置
            cout << "  "; // 清除最上面盘子
        }
        else if (src == 'B' && top_B > 0)
        {
            cct_gotoxy(X_B - 1, Y_B - 2 -
(top_B - 1)); // 定位到最上面盘子的位置
            cout << "  "; // 清除最上面盘子
        }
        else if (src == 'C' && top_C > 0)
        {
            cct_gotoxy(X_C - 1, Y_C - 2 -
(top_C - 1)); // 定位到最上面盘子的位置
            cout << "  "; // 清除最上面盘子
        }
    }
    int plate;
    //从 src 出栈
    if (src == 'A')
        plate = arr_A[--top_A];
    else if (src == 'B')
        plate = arr_B[--top_B];
    else
        plate = arr_C[--top_C];
    // 入栈到 dst
    if (dst == 'A')
        arr_A[top_A++] = plate;
    else if (dst == 'B')
        arr_B[top_B++] = plate;
    else
        arr_C[top_C++] = plate;
    count_step++;
    return plate;
}

void draw_three_columns() //画立柱和底座
{
    int base_color = HDC_COLOR[MAX_LAYER + 1];
//底座和立柱的颜色
    //计算坐标：
    //计算底座和立柱的公共参数
    int base_width = 23 * HDC_Base_Width;   //
底座宽 = 23w
    int base_high = HDC_Base_High;
// 底座高 = h
    int column_w = HDC_Base_Width;
// 立柱宽 = w
    int column_h = 12 * HDC_Base_High;      //
立柱高 = 12h
    //A B C 三个底座左上角坐标
    int baseA_x = HDC_Start_X;
    int baseA_y = HDC_Start_Y;
    int baseB_x = baseA_x +
HDC_Underpan_Distance + base_width;
    int baseB_y = baseA_y;
    int baseC_x = baseB_x +
HDC_Underpan_Distance + base_width;
    int baseC_y = baseA_y;
    //A B C 三个立柱左上角坐标——放在各自底
座的中间
    int colA_x = baseA_x + (base_width -
column_w) / 2;
```

```
    int colA_y = baseA_y - column_h;
    int colB_x = baseB_x + (base_width - column_w) / 2;
    int colB_y = colA_y;
    int colC_x = baseC_x + (base_width - column_w) / 2;
    int colC_y = colA_y;
    //依次画三块底座
    hdc_rectangle(baseA_x, baseA_y, base_width, base_high, base_color);
    Sleep(HDC_Init_Delay);
    hdc_rectangle(baseB_x, baseB_y, base_width, base_high, base_color);
    Sleep(HDC_Init_Delay);
    hdc_rectangle(baseC_x, baseC_y, base_width, base_high, base_color);
    Sleep(HDC_Init_Delay);
    //再画三根立柱
    hdc_rectangle(colA_x, colA_y, column_w, column_h, base_color);
    Sleep(HDC_Init_Delay);
    hdc_rectangle(colB_x, colB_y, column_w, column_h, base_color);
    Sleep(HDC_Init_Delay);
    hdc_rectangle(colC_x, colC_y, column_w, column_h, base_color);
    Sleep(HDC_Init_Delay);
}

void fun1()  //实现菜单 1 中的函数：1.基本解
{
    int layer;
    char src, dst, tmp;
    reset_state();
    layer = input(&src, &dst, &tmp);
    init_state(src, layer);
    count_step = 0;
    hanoi(layer, src, tmp, dst, 1, layer);
}

void fun2()  //实现菜单 2 中的函数：2.基本解(步数记录)
{
    int layer;
    char src, dst, tmp;
    reset_state();
    layer = input(&src, &dst, &tmp);
    init_state(src, layer);
    count_step = 0;
    hanoi(layer, src, tmp, dst, 2, layer);
}

void fun3()  //实现菜单 3 中的函数：3.内部数组显示(横向)
{
    int layer;
    char src, dst, tmp;
    reset_state();
    layer = input(&src, &dst, &tmp);
    init_state(src, layer);
    count_step = 0;
    hanoi(layer, src, tmp, dst, 3, layer);
}

void fun4() //实现菜单 4 中的函数：4.内部数组显示(纵向+横向)
{
    int layer;
    char src, dst, tmp;
    reset_state();
    layer = input(&src, &dst, &tmp);
    init_state(src, layer);
    count_step = 0;
    set_delay();
    cct_cls();
    cct_gotoxy(MenuItem4_Start_X, MenuItem4_Start_Y + 5);
    cout << "从 " << src << " 移动到 " << dst << ", 共 " << layer << " 层, 延时设置为 " << (delay == -1 ? 0 : delay) << " ms";
    cct_gotoxy(MenuItem4_Start_X, MenuItem4_Start_Y);
    cout << "初始: ";
    print_state_row();
    draw_hanoi(4, src);
    perform_delay();
    hanoi(layer, src, tmp, dst, 4, layer);
}
```

```
void fun5() //实现菜单 5 中的函数：5.图形解-预备-画三个圆柱
{
    cct_cls();
    hdc_init(HDC_COLOR[0], HDC_COLOR[MAX_LAYER + 1]); //创建画图环境（类似于 canvas），第一个参数是画布颜色，第二个参数是默认的画笔颜色如果没有设置的话
    hdc_cls();
    draw_three_columns(); //画三个底座和立柱
}

void fun6() //实现菜单 6 中的函数：6.图形解-预备-在起始柱上画 n 个盘子
{
    int layer;
    char src, dst, tmp;
    reset_state();
    layer = input(&src, &dst, &tmp);
    init_state(src, layer);
    cct_cls();
    cct_gotoxy(0, MenuItem4_Start_Y + 5);
    cout << "从 " << src << " 移动到 " << dst << "，共 " << layer << " 层" << endl;
    hdc_init(HDC_COLOR[0], HDC_COLOR[MAX_LAYER + 1]); //创建画图环境（类似于 canvas），第一个参数是画布颜色，第二个参数是默认的画笔颜色如果没有设置的话
    hdc_cls();
    draw_three_columns(); //画三个底座和立柱
    draw_init_disk(layer, src);
    hdc_release(); // 图形句柄释放掉，图还是会留在窗口上
}

void fun7() //实现菜单 7 中的函数：7.图形解-预备-第一次移动
{
    int layer;
    char src, dst, tmp;
    reset_state();
    layer = input(&src, &dst, &tmp);
    init_state(src, layer);
    set_delay();
    cct_cls();
    cct_gotoxy(0, MenuItem8_Start_Y);
    cout << "从 " << src << " 移动到 " << dst << "，共 " << layer << " 层" << endl;

    hdc_init(HDC_COLOR[0], HDC_COLOR[MAX_LAYER + 1], 1400, 900);
    hdc_cls();
    draw_three_columns();
    draw_init_disk(layer, src);

    char first_dst = (layer % 2 == 1) ? dst : tmp;
    move_disk_hdc(1, src, first_dst, layer);
    hdc_release();
}

void fun8() //实现菜单 8 中的函数：8.图形解-自动移动版本
{
    int layer;
    char src, dst, tmp;
    reset_state();
    layer = input(&src, &dst, &tmp);
    init_state(src, layer);
    set_delay();
    cct_cls();
    cct_gotoxy(MenuItem8_Start_X, MenuItem8_Start_Y);
    cout << "初始: ";
    print_state_row();
    draw_hanoi(8, src);
    hdc_init(HDC_COLOR[0], HDC_COLOR[MAX_LAYER + 1], 1400, 900);
    hdc_cls();
    draw_three_columns();
    draw_init_disk(layer, src);
    hanoi(layer, src, tmp, dst, 8, layer);
    hdc_release();
}

void fun9()
{
```

```
    int layer;
    char src, dst, tmp;
    reset_state();
    layer = input(&src, &dst, &tmp);
    init_state(src, layer);
    count_step = 0;
    set_delay();
    cct_cls();
    cct_gotoxy(MenuItem9_Start_X,
MenuItem9_Start_Y);
    cout << "初始: ";
    print_state_row();
    draw_hanoi(8, src);
    hdc_init(HDC_COLOR[0],
HDC_COLOR[MAX_LAYER + 1], 1400, 900);
    hdc_cls();
    draw_three_columns();
    draw_init_disk(layer, src);
    // 游戏主循环
    while (true)
    {
        char move_src, move_dst;
        // 清除提示区域
        cct_gotoxy(0, MenuItem9_Start_Y + 3);
        cout << "
";
        cct_gotoxy(0, MenuItem9_Start_Y + 4);
        cout << "
";
        // 显示输入提示
        cct_gotoxy(0, MenuItem9_Start_Y + 3);
        cin.clear();
        cin.sync();  // 清空输入缓冲区
        cout <<"请输入移动(格式:AC 表示 A->C,
Q-退出): ";
        // 读取输入
        char input[100];
        cin.getline(input, 100);
        // 处理退出
        if ((input[0] =='Q' || input[0] =='q')
&& input[1] == '\0')
        {
            cct_gotoxy(0, MenuItem9_Start_Y
+ 4);
            cout << "游戏退出。共移动了 " <<
count_step << " 步              ";
            Sleep(2000);
            break;
        }
        // 提取有效字符（忽略空格）
        int idx = 0;
        char chars[2] = { 0, 0 };
        for (int i = 0; input[i] !='\0' && idx
< 2; i++)
        {
            if (input[i] !=' ' && input[i] !=
'\t')
            {
                chars[idx++] = input[i];
            }
        }
        // 检查输入长度
        if (idx < 2)
        {
            cct_gotoxy(0, MenuItem9_Start_Y
+ 4);
            cout << "输入格式错误！请输入两
个字母，如: AC                    ";
            Sleep(1500);
            continue;
        }
        // 转换为大写
        move_src = (chars[0] >='a' && chars[0]
<= 'z') ? chars[0] - 'a' + 'A' : chars[0];
        move_dst = (chars[1] >='a' && chars[1]
<= 'z') ? chars[1] - 'a' + 'A' : chars[1];
        // 检查字母是否合法
        if (move_src < 'A' || move_src > 'C'
|| move_dst < 'A' || move_dst > 'C')
        {
            cct_gotoxy(0, MenuItem9_Start_Y
+ 4);
            cout << "输入的字母必须是 A-C!
";
            Sleep(1500);
            continue;
        }
        // 检查源柱和目标柱是否相同
```

```
        if (move_src == move_dst)
        {
            cct_gotoxy(0, MenuItem9_Start_Y + 4);
            cout << "源柱和目标柱不能相同!";
            Sleep(1500);
            continue;
        }
        // 检查移动是否合法
        if (!is_valid_move(move_src, move_dst))
        {
            continue;  // 非法移动，重新输入
        }
        // 获取要移动的盘子号
        int plate = get_top_plate(move_src);
        // 执行移动（图形动画 + 数组更新）
        move_disk_hdc(plate, move_src, move_dst, layer);
        move_plate(move_src, move_dst, 8);
        // 更新文字显示
        cct_gotoxy(MenuItem9_Start_X, MenuItem9_Start_Y);
        show_row(plate, move_src, move_dst);
        cout << "            ";
        draw_hanoi(8, src);
        // 检查是否完成
        if (is_game_over(dst, layer))
        {
            cct_gotoxy(0, MenuItem9_Start_Y + 3);
            cout << "
";
            cct_gotoxy(0, MenuItem9_Start_Y + 4);
            cout << "恭喜！游戏完成！共 " << count_step << " 步";
            // 计算最少步数
            int min_steps = (1 << layer) - 1; // 2^n - 1
            cct_gotoxy(0, MenuItem9_Start_Y + 5);
            cout << "最少需要 " << min_steps << " 步";
            if (count_step == min_steps)
                cout << "你已达到最优解！";
            else
                cout << "还可以优化";
            Sleep(3000);
            break;
        }
    }
    // 释放图形资源
    hdc_release();
}
```